

INTERNSHIP PROJECT REPORT

Windows Registry Change Monitoring System

A Cybersecurity Monitoring and Registry Integrity Tool

Organization	Unified Mentor Pvt. Ltd.
Internship Role	Cybersecurity Intern
Intern	Kushagra Dixit
Internship Period	25 June 2026 - 25 September 2026
Project Type	Individual Internship Project
Primary Technology	Python
Operating Environment	Microsoft Windows

Professional Project Report

Executive Summary

This report presents the work completed during the Cybersecurity Internship at Unified Mentor Pvt. Ltd. The primary project developed during the internship was a Windows Registry Change Monitoring System. The project was independently designed and implemented as a defensive cybersecurity utility for identifying changes to selected Windows Registry locations that may be relevant to persistence, security-policy modification, or script/interpreter execution.

The system establishes a known registry state through a baseline snapshot and compares subsequent snapshots to identify added, modified, or deleted registry values. Detected changes are classified according to simple rule-based indicators, assigned a severity level, recorded as structured events, and made available through a CSV security report. A continuous monitoring mode periodically checks configured registry locations and records detected events.

The implementation is modular and uses Python's standard library, including the Windows Registry API exposed through the winreg module. The project also contains automated unit tests for core change-detection behavior. Functional testing was performed by the developer on a Windows environment.

1. Internship and Project Profile

Item	Details
Organization	Unified Mentor Pvt. Ltd.
Role	Cybersecurity Intern
Internship Duration	25 June 2026 – 25 September 2026
Project	Windows Registry Change Monitoring System
Developer	Kushagra Dixit
Project Ownership	Individually designed and developed
Focus Area	Windows Registry monitoring and defensive cybersecurity

2. Introduction

The Windows Registry is a central configuration database used by Microsoft Windows to store operating-system, application, user, and policy settings. Because registry values can influence application startup and security configuration, selected registry locations can be useful sources of security-relevant change information.

The project developed during the internship focuses on monitoring selected Registry locations and identifying state changes over time. The approach is based on snapshot comparison rather than attempting to determine whether every individual change is malicious. This makes the system a defensive monitoring and integrity-checking tool.

The tool concentrates on locations associated with autorun behavior, Windows Defender and other security-related policies, Winlogon, services, session-manager configuration, and other Windows locations included in the project configuration.

3. Problem Statement

Unauthorized or unexpected changes to Windows Registry values can be difficult to identify through manual inspection. A security analyst needs a repeatable method to establish a known state, detect deviations, record the details of those deviations, and review them later.

The project addresses this problem by providing a lightweight monitoring workflow consisting of baseline creation, integrity checking, continuous monitoring, event classification, logging, and report generation.

4. Project Objectives

- Create a known-good baseline snapshot of selected Windows Registry locations.
- Detect added, modified, and deleted registry values by comparing snapshots.
- Monitor registry locations commonly associated with autorun and persistence mechanisms.
- Identify selected indicators associated with security-policy changes or script/interpreter execution.
- Assign a simple severity level to detected events based on rule-based indicators.
- Support continuous monitoring at a configurable polling interval.
- Record detected events in a structured log for later analysis.
- Generate a CSV report containing the important fields of detected registry events.
- Provide a modular command-line interface for baseline creation, checking, monitoring, and reporting.

5. Scope of the Project

The project scope includes:

- Monitoring configured registry paths under HKCU and HKLM.
- Reading registry values and normalizing binary values for snapshot storage.
- Saving and loading baseline snapshots in JSON format.
- Comparing registry snapshots and producing structured change events.
- Rule-based classification of selected suspicious indicators.
- Periodic monitoring and JSON Lines event logging.
- CSV export for analysis and reporting.
- Automated testing of the core snapshot-difference detector.

The system is not intended to replace a full endpoint detection and response platform, antivirus product, or forensic investigation suite. Its purpose is focused registry change monitoring and defensive analysis.

6. System Requirements

Category	Requirement
Operating System	Microsoft Windows
Language	Python 3
Dependencies	Python standard library only
Registry Interface	Python winreg module
Input	Windows Registry state and optional JSON configuration
Baseline Storage	JSON file
Event Log	JSON Lines (.jsonl)
Report Output	CSV file
Execution	Command-line interface

7. Technology Stack

Technology / Component	Purpose
Python	Core implementation language
winreg	Access to Windows Registry keys and values
argparse	Command-line interface
json	Configuration, baseline, and event serialization
csv	Security report generation
pathlib	File and path handling
dataclasses	Configuration representation
re	Rule-based indicator matching
unittest	Automated testing
time	Periodic monitoring interval

8. System Architecture

The implementation is organized into separate modules so that registry access, detection logic, monitoring, configuration, reporting, and command-line control remain independent.

Module	Responsibility
src/cli.py	Command-line entry point and command dispatch
src/config.py	Default monitored paths and JSON configuration loading

Module	Responsibility
src/registry.py	Windows Registry reading, snapshot creation, and snapshot persistence
src/detector.py	Snapshot comparison, change identification, and severity classification
src/monitor.py	Continuous polling and JSONL event logging
src/reporting.py	Conversion of JSONL events into CSV reports
tests/test_detector.py	Unit tests for core detection behavior

9. System Workflow

- The user creates a baseline using the baseline command.
- The application reads configured Registry locations and stores their current values in a JSON snapshot.
- For an integrity check, a new snapshot is created and compared with the saved baseline.
- For continuous monitoring, the system periodically creates a new snapshot and compares it with the previous state.
- The detector identifies added, modified, or deleted values.
- Each change is passed through rule-based classification to determine reasons and severity.
- Detected events are appended to a JSON Lines log and displayed during continuous monitoring.
- The reporting command converts the event log into a CSV report for analysis.

10. Baseline Creation and Registry Snapshot

The baseline represents a known Registry state against which later states can be compared. The registry module uses the Windows winreg interface to open configured keys and enumerate their values. Each value is stored with its value data and Registry type. Binary values are normalized into a hexadecimal representation so they can be serialized consistently.

```
python -m src.cli baseline
```

The default output is data/baseline.json.

The snapshot process also handles missing Registry keys and permission errors. Missing keys are represented as empty states, while permission errors are recorded as error markers so the snapshot process can continue.

11. Registry Change Detection

The detector compares the union of Registry paths and value names present in the old and new snapshots. The resulting action is determined as follows:

Condition	Detected Action
Value exists in new state but not old state	ADD
Value exists in old state but not new state	DELETE

Condition	Detected Action
Value exists in both states but data differs	MODIFY
Value is unchanged	No event generated

Each event contains a UTC timestamp, action, Registry path, value name, old value, new value, severity, and detection reasons.

12. Suspicious Change Classification

The project uses rule-based classification rather than claiming that every Registry change is malicious. The detector checks the changed path, value name, and value content for selected indicators.

Indicator	Detection Behavior
Executable autorun entry	Flags executable extensions under Run or RunOnce locations.
Security-policy/protection indicator	Checks for terms associated with disabling or changing protection/security policy.
Script/interpreter indicator	Checks for PowerShell, WScript, CScript, or MSHTA references.
No selected indicator	Records the event as a Registry state change with LOW severity.

Events matching one or more of the selected indicators are assigned HIGH severity by the current implementation. Other detected Registry changes are assigned LOW severity.

13. Monitored Registry Locations

The default configuration includes the following Registry locations:

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run
- HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce
- HKLM\Software\Microsoft\Windows\CurrentVersion\Run
- HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce
- HKLM\SOFTWARE\Policies\Microsoft\Windows Defender
- HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System
- HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer
- HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
- HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options
- HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Drivers32
- HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows
- HKLM\SYSTEM\CurrentControlSet\Services
- HKLM\SYSTEM\CurrentControlSet\Control\Session Manager

14. Continuous Monitoring

The monitoring engine supports periodic Registry snapshot comparison. The default polling interval is 10 seconds, and the interval can be overridden through the command-line interface or JSON configuration.

```
python -m src.cli monitor
```

When changes are detected, the system appends events to a JSON Lines log and prints a concise event summary containing severity, action, Registry path, and value name. Monitoring continues until the user stops the process.

15. Logging and Reporting

Detected events are stored as JSON objects, one event per line. The reporting module converts these records into a CSV file suitable for review and further analysis.

Field	Purpose
timestamp	Time at which the event was generated
action	ADD, MODIFY, or DELETE
path	Affected Registry path
name	Affected Registry value name
old_value	Value before the change
new_value	Value after the change
severity	HIGH or LOW according to current rules
reasons	Reason(s) associated with the classification

```
python -m src.cli report
```

16. Command-Line Interface

Command	Purpose	Default Output / Input
baseline	Create baseline snapshot	data/baseline.json
check	Compare current state with baseline	logs/registry_changes.jsonl
monitor	Continuously monitor configured paths	logs/registry_changes.jsonl
report	Generate CSV report from event log	logs/registry_report.csv

The CLI also supports an optional configuration file, allowing the polling interval and monitored Registry paths to be customized.

17. Testing and Validation

The project was tested by the developer in a Windows environment. In addition, the repository includes automated unit tests for the core detection logic.

Test Case	Expected Behavior
Added autorun value	Detected as ADD and classified HIGH when the value contains an executable indicator.
Modified value	Detected as MODIFY.
Unchanged state	Produces no detection event.

The automated test suite contains three detector tests covering these cases. The tests focus on the comparison and classification layer rather than attempting to emulate the entire Windows Registry.

18. Key Implementation Outcomes

- A working Windows-specific Registry snapshot mechanism was implemented.
- Baseline and subsequent snapshots can be compared consistently.
- Added, modified, and deleted values are distinguished automatically.
- Selected persistence and security-related indicators are highlighted through rule-based classification.
- Continuous monitoring is supported through periodic polling.
- Detected events can be retained in JSONL form and exported to CSV.
- Configuration is separated from detection and monitoring logic.
- The core detection behavior is covered by automated unit tests.
- The project was implemented independently as an individual internship project.

19. Development Challenges and Solutions

Challenge	Approach Used
Reading Windows-specific Registry data	Used Python's winreg module with explicit Windows environment validation.
Comparing nested Registry states	Represented snapshots as structured dictionaries and compared paths and value names.
Handling missing keys	Converted missing-key conditions into empty Registry states.
Handling access restrictions	Captured permission errors as snapshot error markers.
Identifying potentially important changes	Implemented rule-based checks for autorun, security-policy, and interpreter indicators.
Maintaining useful event history	Stored events in JSON Lines and provided CSV export.
Making the monitor configurable	Added JSON configuration and a command-line polling interval override.

20. Skills and Knowledge Gained

- Practical understanding of Windows Registry structure and security relevance.
- Python development using operating-system APIs and standard-library modules.
- Designing modular cybersecurity utilities.
- Implementing baseline and integrity-comparison logic.
- Rule-based security event classification.
- Command-line application design.
- Structured logging and CSV reporting.
- Writing and running unit tests.
- Handling platform-specific errors and permission conditions.
- Applying defensive cybersecurity concepts to a practical monitoring problem.

21. Current Limitations

- The tool is Windows-specific because it relies on the winreg module.
- Detection is rule-based and therefore should be treated as an indicator rather than definitive proof of malicious activity.
- The current monitor uses periodic polling rather than Windows event-driven Registry notifications.
- The project monitors configured locations rather than the complete Windows Registry.
- The current implementation does not provide a graphical user interface.
- The project is intended as a focused monitoring utility and does not provide the broader telemetry of a full endpoint security platform.

22. Future Scope

- Add a graphical dashboard for live event visualization.
- Introduce configurable severity rules and user-defined detection signatures.
- Add stronger correlation between Registry changes and process execution telemetry.
- Integrate notifications such as email, webhook, or security-platform alerts.
- Support richer filtering and searching of historical events.
- Add cryptographic integrity protection for baseline and event files.
- Expand automated tests to cover more Registry data types and error conditions.
- Explore event-driven monitoring mechanisms to reduce polling overhead.

23. Conclusion

The Windows Registry Change Monitoring System was developed as an individual cybersecurity project during the internship at Unified Mentor Pvt. Ltd. The project provides a structured approach to monitoring selected Windows Registry locations through baseline creation, snapshot comparison, change detection, rule-based classification, continuous monitoring, event logging, and CSV reporting.

The implementation demonstrates practical application of Python programming, Windows system interaction, defensive security monitoring, modular software design, and testing. The resulting tool provides a focused foundation for identifying Registry changes that may warrant further security investigation.

24. References

- Project source code and README provided as part of the internship project.
- Python standard-library modules used by the implementation: winreg, argparse, json, csv, pathlib, dataclasses, re, time, and unittest.
- Windows Registry concepts and security-relevant Registry locations as implemented in the project configuration.

Appendix A - Project Structure

```
windows-registry/  
├── src/  
│   ├── cli.py  
│   ├── config.py  
│   ├── detector.py  
│   ├── monitor.py  
│   ├── registry.py  
│   └── reporting.py  
├── data/  
├── logs/  
├── tests/  
│   └── test_detector.py  
├── config.example.json  
├── requirements.txt  
└── README.md
```

Appendix B - Typical Usage

```
python -m src.cli baseline  
python -m src.cli check  
python -m src.cli monitor  
python -m src.cli report
```